

MY LEETCODE

# REVISION HANDBOOK

Patterns • Complexity • Solutions • Recall

## 31 problems

Built for active recall before interviews — understand the pattern, then rebuild the code.

**SMALL STEPS. REPEATED RECALL. STRONGER DSA.**

# Revision Index

Find a problem by number, pattern, or difficulty.

| NO.  | PROBLEM                                                            | PATTERN                 | LEVEL  |
|------|--------------------------------------------------------------------|-------------------------|--------|
| 0001 | <b>1. Two Sum</b>                                                  | Hashing / Set           | Easy   |
| 0002 | <b>2. Add Two Numbers</b>                                          | Linked List             | Medium |
| 0003 | <b>3. Longest Substring Without Repeating Characters</b>           | Hashing / Set           | Medium |
| 0004 | <b>4. Median of Two Sorted Arrays</b>                              | Sorting                 | Hard   |
| 0007 | <b>7. Reverse Integer</b>                                          | Math / Simulation       | Medium |
| 0009 | <b>9. Palindrome Number</b>                                        | Math / Simulation       | Easy   |
| 0011 | <b>11. Container With Most Water</b>                               | Two Pointers            | Medium |
| 0014 | <b>14. Longest Common Prefix</b>                                   | Arrays / Implementation | Easy   |
| 0019 | <b>19. Remove Nth Node From End of List</b>                        | Linked List             | Medium |
| 0021 | <b>21. Merge Two Sorted Lists</b>                                  | Linked List             | Easy   |
| 0024 | <b>24. Swap Nodes in Pairs</b>                                     | Linked List             | Medium |
| 0033 | <b>33. Search in Rotated Sorted Array</b>                          | Arrays / Implementation | Medium |
| 0034 | <b>34. Find First and Last Position of Element in Sorted Array</b> | Binary Search           | Medium |
| 0035 | <b>35. Search Insert Position</b>                                  | Binary Search           | Easy   |
| 0049 | <b>49. Group Anagrams</b>                                          | Hashing / Set           | Medium |
| 0053 | <b>53. Maximum Subarray</b>                                        | Greedy / Kadane         | Medium |
| 0076 | <b>76. Minimum Window Substring</b>                                | Hashing / Set           | Hard   |
| 0081 | <b>81. Search in Rotated Sorted Array II</b>                       | Binary Search           | Medium |
| 0094 | <b>94. Binary Tree Inorder Traversal</b>                           | Binary Search           | Easy   |
| 0098 | <b>98. Validate Binary Search Tree</b>                             | Binary Search           | Medium |
| 0100 | <b>100. Same Tree</b>                                              | Binary Search           | Easy   |
| 0101 | <b>101. Symmetric Tree</b>                                         | Binary Search           | Easy   |
| 0121 | <b>121. Best Time to Buy and Sell Stock</b>                        | Greedy / Kadane         | Easy   |
| 0122 | <b>122. Best Time to Buy and Sell Stock II</b>                     | Greedy / Kadane         | Medium |
| 0128 | <b>128. Longest Consecutive Sequence</b>                           | Hashing / Set           | Medium |
| 0189 | <b>189. Rotate Array</b>                                           | Math / Simulation       | Medium |
| 0209 | <b>209. Minimum Size Subarray Sum</b>                              | Arrays / Implementation | Medium |
| 0215 | <b>215. Kth Largest Element in an Array</b>                        | Arrays / Implementation | Medium |
| 0239 | <b>239. Sliding Window Maximum</b>                                 | Two Pointers            | Hard   |
| 0455 | <b>455. Assign Cookies</b>                                         | Sorting                 | Easy   |
| 3718 | <b>4080. Smallest Missing Multiple of K</b>                        | Hashing / Set           | Easy   |

## #0001

PROBLEM

### 1. Two Sum

DIFFICULTY

Easy

PATTERN

Hashing / Set

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

PROBLEM 0001

# 1. Two Sum

Easy

Hashing / Set

C++

## QUESTION



Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`.



### Example

Input: `nums = [2,7,11,15]`, `target = 9` Output: `[0,1]` Explanation: Because `nums[0] + nums[1] == 9`, we return `[0, 1]`.



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**$O(n)$  avg.**



SPACE COMPLEXITY

**$O(n)$**



PATTERN

**Hashing / Set**

`</>` **Solution (C++)**

Problem 0001

```
class Solution {
public:
    vector<int> twoSum(vector<int>& nums, int target) {
        unordered_map<int, int> mp;
        for (int i = 0; i < nums.size(); i++) {
            int x = target - nums[i];
            if (mp.find(x) != mp.end()) {
                return {mp[x], i};
            }
            mp[nums[i]] = i;
        }
        return {};
    }
};
```

## #0002

PROBLEM

### 2. Add Two Numbers

DIFFICULTY

Medium

PATTERN

Linked List

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0002

## 2. Add Two Numbers

Medium

Linked List

C++

### QUESTION

- ✓ You are given two non-empty linked lists representing two non-negative integers. The digits are stored in reverse order, and each of their nodes contains a single digit. Add the two numbers and return the sum as a linked list.

#### Example

Input: l1 = [2,4,3], l2 = [5,6,4] Output: [7,0,8] Explanation: 342 + 465 = 807.

#### Visual Representation



pattern → nodes → result

✓ TIME COMPLEXITY  
**O(n)**

✓ SPACE COMPLEXITY  
**O(1)**

✓ PATTERN  
**Linked List**

### </> Solution (C++)

Problem 0002

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* addTwoNumbers(ListNode* l1, ListNode* l2) {
        ListNode* dummy = new ListNode(0);
        ListNode* temp = dummy;
        int carry = 0;
        while (l1 != NULL || l2 != NULL || carry != 0) {
            int sum = carry;
            if (l1 != NULL) {
                sum += l1->val;
                l1 = l1->next;
            }
            if (l2 != NULL) {
                sum += l2->val;
                l2 = l2->next;
            }
            carry = sum / 10;
            temp->next = new ListNode(sum % 10);
            temp = temp->next;
        }
        return dummy->next;
    }
};
```

PROBLEM 0003

## #0003

PROBLEM

## 3. Longest Substring Without Repeating Characters

DIFFICULTY

Medium

PATTERN

Hashing / Set

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 3. Longest Substring Without Repeating Characters

Medium

Hashing / Set

C++

## QUESTION



Given a string  $s$ , find the length of the longest substring without duplicate characters.



## Example

Input:  $s = \text{"abcabcbb"}$  Output: 3 Explanation: The answer is "abc", with the length of 3. Note that "bca" and "cab" are also correct answers.



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 $O(n)$  avg.

SPACE COMPLEXITY

 $O(n)$ 

PATTERN

Hashing / Set

## &lt;/&gt; Solution (C++)

Problem 0003

```
class Solution {
public:
    int lengthOfLongestSubstring(string s) {
        unordered_set<char> st;
        int left = 0;
        int maxi = 0;
        for (int right = 0; right < s.size(); right++) {
            while (st.find(s[right]) != st.end()) {
                st.erase(s[left]);
                left++;
            }
            st.insert(s[right]);
            maxi = max(maxi, right - left + 1);
        }
        return maxi;
    }
};
```

## #0004

PROBLEM

### 4. Median of Two Sorted Arrays

DIFFICULTY

Hard

PATTERN

Sorting

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0004

# 4. Median of Two Sorted Arrays

Hard

Sorting

C++

## QUESTION



Given two sorted arrays `nums1` and `nums2` of size `m` and `n` respectively, return the median of the two sorted arrays.



### Example

Input: `nums1 = [1,3]`, `nums2 = [2]` Output: `2.00000` Explanation: merged array = `[1,2,3]` and median is `2`.



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**$O(n \log n)$**



SPACE COMPLEXITY

**$O(n) / O(1)$**



PATTERN

**Sorting**

</> Solution (C++)

Problem 0004

```
class Solution {
public:
    double findMedianSortedArrays(vector<int>& nums1, vector<int>& nums2) {
        nums1.insert(nums1.end(), nums2.begin(), nums2.end());
        sort(nums1.begin(), nums1.end());
        if(nums1.size()%2==0){
            return (nums1[(nums1.size()/2)-1]+nums1[nums1.size()/2])/2.0;
        }
        else{
            return nums1[nums1.size()/2.0];
        }
        return{};
    }
};
```

PROBLEM 0007

## #0007

PROBLEM

## 7. Reverse Integer

DIFFICULTY

Medium

PATTERN

Math / Simulation

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

## 7. Reverse Integer

Medium

Math / Simulation

C++

## QUESTION



Given a signed 32-bit integer  $x$ , return  $x$  with its digits reversed. If reversing  $x$  causes the value to go outside the signed 32-bit integer range  $[-2^{31}, 2^{31} - 1]$ , then return 0.



## Example

Input:  $x = 123$  Output: 321



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 $O(n)$ 

SPACE COMPLEXITY

 $O(1)$ 

PATTERN

Math / Simulation

&lt;/&gt; Solution (C++)

Problem 0007

```
class Solution {
public:
    int reverse(int x) {
        int rev = 0;

        while(x!=0){
            int d = x%10;
            if(rev > INT_MAX/10 || rev < INT_MIN/10)
                return 0;
            rev = rev*10 + d;
            x=x/10;
        }
        return rev;
    }
};
```

PROBLEM 0009

#0009

PROBLEM

## 9. Palindrome Number

DIFFICULTY

Easy

PATTERN

Math / Simulation

LANGUAGE

C++

QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

# 9. Palindrome Number

Easy

Math / Simulation

C++

### QUESTION



Given an integer  $x$ , return true if  $x$  is a palindrome, and false otherwise.



### Example

Input:  $x = 121$  Output: true Explanation: 121 reads as 121 from left to right and from right to left.



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 $O(n)$ 

SPACE COMPLEXITY

 $O(1)$ 

PATTERN

Math / Simulation

&lt;/&gt; Solution (C++)

Problem 0009

```
class Solution {
public:
    bool isPalindrome(int x){
        if (x < 0)
            return false;
        if (x != 0 && x % 10 == 0)
            return false;
        int rev = 0;
        while (x > rev) {
            rev = rev * 10 + x % 10;
            x /= 10;
        }
        if(x==rev || x==rev/10)
            return true;
        else
            return false;
    }
};
```

PROBLEM 0011

## #0011

PROBLEM

## 11. Container With Most Water

DIFFICULTY

Medium

PATTERN

Two Pointers

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 11. Container With Most Water

Medium

Two Pointers

C++

## QUESTION



You are given an integer array height of length n. There are n vertical lines drawn such that the two endpoints of the ith line are (i, 0) and (i, height[i]).



## Example

Input: height = [1,8,6,2,5,4,8,3,7] Output: 49  
Explanation: The above vertical lines are represented by array [1,8,6,2,5,4,8,3,7]. In this case, the max area of water (blue section) the container can contain is 49.



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**O(n)**

SPACE COMPLEXITY

**O(1)**

PATTERN

**Two Pointers**

## &lt;/&gt; Solution (C++)

Problem 0011

```
class Solution {
public:
    int maxArea(vector<int>& height) {
        int left = 0;
        int right = height.size()-1;
        int area = 0;
        while(left<right){
            int h = min(height[left],height[right]);
            int a = h*(right-left);
            area = max(a,area);
            if(height[left]<height[right]){
                left++;
            }
            else{
                right--;
            }
        }
        return area;
    }
};
```

## #0014

PROBLEM

### 14. Longest Common Prefix

DIFFICULTY

Easy

PATTERN

Arrays / Implementa...

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0014

# 14. Longest Common Prefix

Easy

Arrays / Implementation

C++

## QUESTION



Write a function to find the longest common prefix string amongst an array of strings.



### Example

Input: strs = ["flower","flow","flight"]  
Output: "fl"



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**O(n)**



SPACE COMPLEXITY

**O(1)**



PATTERN

Arrays / Implementation

## </> Solution (C++)

Problem 0014

```
class Solution {
public:
    string longestCommonPrefix(vector<string>& strs) {

        string prefix = strs[0];

        for(int i = 1; i < strs.size(); i++) {

            int j = 0;

            while(j < prefix.size() &&
                  j < strs[i].size() &&
                  prefix[j] == strs[i][j]) {
                j++;
            }

            prefix = prefix.substr(0, j);

            if(prefix == "")
                return "";
        }

        return prefix;
    }
};
```

## PROBLEM 0019

## #0019

PROBLEM

## 19. Remove Nth Node From End of List

DIFFICULTY

Medium

PATTERN

Linked List

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 19. Remove Nth Node From End of List

Medium

Linked List

C++

## QUESTION



Given the head of a linked list, remove the nth node from the end of the list and return its head.



## Example

Input: head = [1,2,3,4,5], n = 2 Output: [1,2,3,5]



## Visual Representation



pattern → nodes → result



TIME COMPLEXITY

**O(n)**

SPACE COMPLEXITY

**O(1)**

PATTERN

**Linked List**

## &lt;/&gt; Solution (C++)

Problem 0019

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* removeNthFromEnd(ListNode* head, int n) {
        ListNode *dummy = new ListNode(0);
        dummy->next = head;
        ListNode *temp=head;
        int count = 0;
        while(temp!=NULL){
            temp = temp->next;
            count++;
        }
        int k=count-n;
        temp = dummy;
        while(k>0){
            temp=temp->next;
            k--;
        }
        ListNode *temp1=temp->next;
        temp->next=temp1->next;
        delete temp1;
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};

```

## #0021

PROBLEM

### 21. Merge Two Sorted Lists

DIFFICULTY

Easy

PATTERN

Linked List

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0021

# 21. Merge Two Sorted Lists

Easy

Linked List

C++

## QUESTION

✓ You are given the heads of two sorted linked lists list1 and list2.

### Example

Input: list1 = [1,2,4], list2 = [1,3,4] Output: [1,1,2,3,4,4]

### Visual Representation



pattern → nodes → result

✓ TIME COMPLEXITY  
**O(n)**

✓ SPACE COMPLEXITY  
**O(1)**

✓ PATTERN  
**Linked List**

## </> Solution (C++)

Problem 0021

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* mergeTwoLists(ListNode* list1, ListNode* list2) {
        if(list1==NULL){return list2;}
        if(list2==NULL){return list1;}
        ListNode* dummy=new ListNode(-1);
        ListNode* temp = dummy;
        while(list1!=NULL && list2!=NULL){
            if(list1->val<=list2->val){
                temp->next=list1;
                list1=list1->next;
            }
            else{
                temp->next=list2;
                list2=list2->next;
            }
            temp=temp->next;
        }
        if(list1!=NULL){
            temp->next=list1;
        }
        else{
            temp->next=list2;
        }
        return dummy->next;
    }
};
```

## #0024

PROBLEM

## 24. Swap Nodes in Pairs

DIFFICULTY

Medium

PATTERN

Linked List

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 24. Swap Nodes in Pairs

Medium

Linked List

C++

## QUESTION

- ✓ Given a linked list, swap every two adjacent nodes and return its head. You must solve the problem without modifying the values in the list's nodes (i.e., only nodes themselves may be changed.)

## Example

Input → Output

## Visual Representation



✓ TIME COMPLEXITY  
**O(n)**

✓ SPACE COMPLEXITY  
**O(1)**

✓ PATTERN  
**Linked List**

## &lt;/&gt; Solution (C++)

Problem 0024

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode() : val(0), next(nullptr) {}
 *     ListNode(int x) : val(x), next(nullptr) {}
 *     ListNode(int x, ListNode *next) : val(x), next(next) {}
 * };
 */
class Solution {
public:
    ListNode* swapPairs(ListNode* head) {
        ListNode* dummy = new ListNode(0);
        dummy->next = head;
        ListNode* temp = dummy;
        while(temp->next != NULL && temp->next->next != NULL) {
            ListNode* first = temp->next;
            ListNode* second = first->next;
            first->next = second->next;
            second->next = first;
            temp->next = second;
            temp = first;
        }
        ListNode* newHead = dummy->next;
        delete dummy;
        return newHead;
    }
};

```

PROBLEM 0033

# 33. Search in Rotated Sorted Array

Medium

Arrays / Implementation

C++

## QUESTION

✓ There is an integer array `nums` sorted in ascending order (with distinct values).

### Example

Input: `nums = [4,5,6,7,0,1,2]`, `target = 0`  
Output: 4

### Visual Representation

INPUT

PATTERN

OUTPUT

✓ TIME COMPLEXITY  
 **$O(n)$**

✓ SPACE COMPLEXITY  
 **$O(1)$**

✓ PATTERN  
**Arrays / Implementation**

</> Solution (C++)

Problem 0033

```
class Solution {
public:
    int search(vector<int>& nums, int target) {
        for(int i=0;i<nums.size();i++){
            if(nums[i]==target){
                return i;
            }
        }
        return -1;
    }
};
```

#0033

PROBLEM

## 33. Search in Rotated Sorted Array

DIFFICULTY

Medium

PATTERN

Arrays / Implementa...

LANGUAGE

C++

### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## #0034

PROBLEM

### 34. Find First and Last Position of Element in Sorted Array

DIFFICULTY

Medium

PATTERN

Binary Search

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0034

# 34. Find First and Last Position of Element in Sorted Array

Medium

Binary Search

C++

## QUESTION

- ✓ Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

### Example

Input: `nums = [5,7,7,8,8,10]`, `target = 8`  
Output: `[3,4]`

### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**$O(n)$**



SPACE COMPLEXITY

**$O(1)$**



PATTERN

**Binary Search**

## </> Solution (C++)

Problem 0034

```
class Solution {
public:
    vector<int> searchRange(vector<int>& nums, int target) {
        int first = -1;
        int last = -1;
        // Find first occurrence
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                first = mid;
                right = mid - 1; // search left
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        // Find last occurrence
        left = 0;
        right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target) {
                last = mid;
                left = mid + 1; // search right
            }
            else if (target > nums[mid]) {
                left = mid + 1;
            }
            else {
                right = mid - 1;
            }
        }
        return {first, last};
    }
};
```

## #0035

PROBLEM

### 35. Search Insert Position

DIFFICULTY

Easy

PATTERN

Binary Search

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0035

# 35. Search Insert Position

Easy

Binary Search

C++

## QUESTION

- ✓ Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

### Example

Input: nums = [1,3,5,6], target = 5 Output: 2

### Visual Representation

INPUT

PATTERN

OUTPUT

✓ TIME COMPLEXITY  
 **$O(n)$**

✓ SPACE COMPLEXITY  
 **$O(1)$**

✓ PATTERN  
**Binary Search**

## </> Solution (C++)

Problem 0035

```
class Solution {
public:
    int searchInsert(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size()-1;
        while(left<=right){
            int mid = left + (right - left)/2;
            if(nums[mid]==target){
                return mid;
            }
            if(target>nums[mid]){
                left=mid+1;
            }
            else{
                right=mid-1;
            }
        }
        return left;
    }
};
```

PROBLEM 0049

## #0049

PROBLEM

## 49. Group Anagrams

DIFFICULTY

Medium

PATTERN

Hashing / Set

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 49. Group Anagrams

Medium

Hashing / Set

C++

## QUESTION



Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.



## Example

Input → Output



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 $O(n \log n)$ 

SPACE COMPLEXITY

 $O(n) / O(1)$ 

PATTERN

Hashing / Set

## &lt;/&gt; Solution (C++)

Problem 0049

```
class Solution {
public:
    vector<vector<string>> groupAnagrams(vector<string>& strs) {
        unordered_map<string, vector<string>> mp;
        for(string s: strs){
            string key = s;
            sort(key.begin(), key.end());
            mp[key].push_back(s);
        }
        vector<vector<string>> ans;
        for(auto x: mp){
            ans.push_back(x.second);
        }
        return ans;
    }
};
```

PROBLEM 0053

## #0053

PROBLEM

## 53. Maximum Subarray

DIFFICULTY

Medium

PATTERN

Greedy / Kadane

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 53. Maximum Subarray

Medium

Greedy / Kadane

C++

## QUESTION



Given an integer array `nums`, find the subarray with the largest sum, and return its sum.



## Example

Input: `nums = [-2,1,-3,4,-1,2,1,-5,4]` Output: 6  
 Explanation: The subarray `[4,-1,2,1]` has the largest sum 6.



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 **$O(n)$** 

SPACE COMPLEXITY

 **$O(1)$** 

PATTERN

**Greedy / Kadane**

&lt;/&gt; Solution (C++)

Problem 0053

```
class Solution {
public:
    int maxSubArray(vector<int>& nums) {
        int sum = 0;
        int maxsum = INT_MIN;
        for(int i = 0; i < nums.size(); i++){
            sum += nums[i];
            maxsum = max(maxsum, sum);
            if(sum < 0){
                sum = 0;
            }
        }
        return maxsum;
    }
};
```

## #0076

PROBLEM

### 76. Minimum Window Substring

DIFFICULTY

Hard

PATTERN

Hashing / Set

LANGUAGE

C++

QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0076

# 76. Minimum Window Substring

Hard

Hashing / Set

C++

## QUESTION

- ✓ Given two strings  $s$  and  $t$  of lengths  $m$  and  $n$  respectively, return the minimum window substring of  $s$  such that every character in  $t$  (including duplicates) is included in the window. If there is no such substring, return the empty string  $""$ .

### Example

Input:  $s = \text{"ADOBECODEBANC"}$ ,  $t = \text{"ABC"}$  Output:  $\text{"BANC"}$  Explanation: The minimum window substring  $\text{"BANC"}$  includes 'A', 'B', and 'C' from string  $t$ .

### Visual Representation

INPUT

PATTERN

OUTPUT

✓ TIME COMPLEXITY  
 **$O(n)$  avg.**

✓ SPACE COMPLEXITY  
 **$O(n)$**

✓ PATTERN  
**Hashing / Set**

</> Solution (C++)

Problem 0076

```
class Solution {
public:
    string minWindow(string s, string t) {
        unordered_map<char,int> n,w;
        for(char c:t) n[c]++;
        int l = 0,start=0;
        int req = n.size();
        int have = 0;
        int len = INT_MAX ;
        for(int r = 0; r<s.size(); r++){
            w[s[r]]++;
            if(n.count(s[r]) && w[s[r]]==n[s[r]]){
                have++;
            }
            while(have == req){
                if(r-l+1 < len){
                    len = r-l+1;
                    start = l;
                }
                w[s[l]]--;
                if(n.count(s[l])&&w[s[l]]<n[s[l]]){
                    have--;
                }
                l++;
            }
        }
        if(len == INT_MAX) return "";
        return s.substr(start,len);
    }
};
```

## #0081

PROBLEM

### 81. Search in Rotated Sorted Array II

DIFFICULTY

Medium

PATTERN

Binary Search

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0081

# 81. Search in Rotated Sorted Array II

Medium

Binary Search

C++

## QUESTION



There is an integer array `nums` sorted in non-decreasing order (not necessarily with distinct values).



### Example

Input: `nums = [2,5,6,0,0,1,2]`, `target = 0`  
Output: `true`



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**$O(n)$**



SPACE COMPLEXITY

**$O(1)$**



PATTERN

**Binary Search**

## </> Solution (C++)

Problem 0081

```
class Solution {
public:
    bool search(vector<int>& nums, int target) {
        int left = 0;
        int right = nums.size() - 1;
        while (left <= right) {
            int mid = left + (right - left) / 2;
            if (nums[mid] == target)
                return true;
            // Duplicates: cannot determine which half is sorted
            if (nums[left] == nums[mid] && nums[mid] == nums[right]) {
                left++;
                right--;
            }
            // Left half is sorted
            else if (nums[left] <= nums[mid]) {
                if (nums[left] <= target && target < nums[mid])
                    right = mid - 1;
                else
                    left = mid + 1;
            }
            // Right half is sorted
            else {
                if (nums[mid] < target && target <= nums[right])
                    left = mid + 1;
                else
                    right = mid - 1;
            }
        }
        return false;
    }
};
```

PROBLEM 0094

# 94. Binary Tree Inorder Traversal

Easy

Binary Search

C++

## QUESTION

Given the root of a binary tree, return the inorder traversal of its nodes' values.

### Example

Input → Output

### Visual Representation

INPUT

PATTERN

OUTPUT

TIME COMPLEXITY  
 **$O(n)$**

SPACE COMPLEXITY  
 **$O(1)$**

PATTERN  
**Binary Search**

## </> Solution (C++)

Problem 0094

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    vector<int> ans;
    vector<int> inorderTraversal(TreeNode* root) {
        if(root == NULL)
            return ans;
        inorderTraversal(root->left);
        ans.push_back(root->val);
        inorderTraversal(root->right);
        return ans;
    }
};
```

#0094

PROBLEM

## 94. Binary Tree Inorder Traversal

DIFFICULTY

Easy

PATTERN

Binary Search

LANGUAGE

C++

### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## #0098

PROBLEM

### 98. Validate Binary Search Tree

DIFFICULTY

Medium

PATTERN

Binary Search

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0098

# 98. Validate Binary Search Tree

Medium

Binary Search

C++

## QUESTION

- ✓ Given the root of a binary tree, determine if it is a valid binary search tree (BST).

### Example

Input: root = [2,1,3] Output: true

### Visual Representation

INPUT

PATTERN

OUTPUT

✓ TIME COMPLEXITY  
**O(n)**

✓ SPACE COMPLEXITY  
**O(1)**

✓ PATTERN  
**Binary Search**

## </> Solution (C++)

Problem 0098

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool check(TreeNode* root, long long low, long long high){
        if(root == NULL) return true;
        if (root->val <= low || root->val >= high)
            return false;
        return check(root->left, low, root->val)&&check(root->right, root->val, high);
    }
    bool isValidBST(TreeNode* root) {
        return check(root, LLONG_MIN, LLONG_MAX);
    }
};
```

## #0100

PROBLEM

### 100. Same Tree

DIFFICULTY

Easy

PATTERN

Binary Search

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

PROBLEM 0100

# 100. Same Tree

Easy

Binary Search

C++

## QUESTION



Given the roots of two binary trees p and q, write a function to check if they are the same or not.



### Example

Input: p = [1,2,3], q = [1,2,3] Output: true



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**O(n)**



SPACE COMPLEXITY

**O(1)**



PATTERN

**Binary Search**

## </> Solution (C++)

Problem 0100

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool isSameTree(TreeNode* p, TreeNode* q) {
        if(p==NULL && q==NULL) return true;
        if(p==NULL || q==NULL) return false;
        if(p->val!=q->val) return false;
        return isSameTree(p->left,q->left)&&isSameTree(p->right,q->right);
    }
};
```

## #0101

PROBLEM

### 101. Symmetric Tree

DIFFICULTY

Easy

PATTERN

Binary Search

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

PROBLEM 0101

# 101. Symmetric Tree

Easy

Binary Search

C++

## QUESTION



Given the root of a binary tree, check whether it is a mirror of itself (i.e., symmetric around its center).



### Example

Input: root = [1,2,2,3,4,4,3] Output: true



### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**O(n)**



SPACE COMPLEXITY

**O(1)**



PATTERN

**Binary Search**

## </> Solution (C++)

Problem 0101

```
/**
 * Definition for a binary tree node.
 * struct TreeNode {
 *     int val;
 *     TreeNode *left;
 *     TreeNode *right;
 *     TreeNode() : val(0), left(nullptr), right(nullptr) {}
 *     TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
 *     TreeNode(int x, TreeNode *left, TreeNode *right) : val(x), left(left), right(right) {}
 * };
 */
class Solution {
public:
    bool mirror(TreeNode* left, TreeNode* right) {
        if (left == NULL && right == NULL)
            return true;
        if (left == NULL || right == NULL)
            return false;
        if (left->val != right->val)
            return false;
        return mirror(left->left, right->right) &&
            mirror(left->right, right->left);
    }
    bool isSymmetric(TreeNode* root) {
        if (root == NULL)
            return true;
        return mirror(root->left, root->right);
    }
};
```

## PROBLEM 0121

## #0121

PROBLEM

## 121. Best Time to Buy and Sell Stock

DIFFICULTY

Easy

PATTERN

Greedy / Kadane

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

# 121. Best Time to Buy and Sell Stock

Easy

Greedy / Kadane

C++

## QUESTION



You are given an array prices where prices[i] is the price of a given stock on the ith day.



## Example

Input: prices = [7,1,5,3,6,4] Output: 5  
Explanation: Buy on day 2 (price = 1) and sell on day 5 (price = 6), profit = 6-1 = 5. Note that buying on day 2 and selling on day 1 is not allowed because you must buy before you sell.



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**O(n)**

SPACE COMPLEXITY

**O(1)**

PATTERN

**Greedy / Kadane**

## &lt;/&gt; Solution (C++)

Problem 0121

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int minprice = prices[0];
        int profit = 0;
        for(int i =1;i<prices.size();i++){
            minprice= min(minprice,prices[i]);
            profit = max(profit,prices[i]-minprice);
        }
        return profit;
    }
};
```

## PROBLEM 0122

## #0122

PROBLEM

## 122. Best Time to Buy and Sell Stock II

DIFFICULTY

Medium

PATTERN

Greedy / Kadane

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

[Pattern](#) → [recall](#) → [re-implement](#)

## 122. Best Time to Buy and Sell Stock II

Medium

Greedy / Kadane

C++

## QUESTION



You are given an integer array prices where prices[i] is the price of a given stock on the ith day.



## Example

Input: prices = [7,1,5,3,6,4] Output: 7  
Explanation: Buy on day 2 (price = 1) and sell on day 3 (price = 5), profit = 5-1 = 4. Then buy on day 4 (price = 3) and sell on day 5 (price = 6), profit = 6-3 = 3. Total profit is 4 + 3 = 7.



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**O(n)**

SPACE COMPLEXITY

**O(1)**

PATTERN

**Greedy / Kadane**

## &lt;/&gt; Solution (C++)

Problem 0122

```
class Solution {
public:
    int maxProfit(vector<int>& prices) {
        int profit=0;
        for(int i = 1;i<prices.size();i++){
            if(prices[i]>prices[i-1]){
                profit = profit + prices[i] - prices[i-1];
            }
        }
        return profit;
    }
};
```

## #0128

PROBLEM

## 128. Longest Consecutive Sequence

DIFFICULTY

Medium

PATTERN

Hashing / Set

LANGUAGE

C++

## 128. Longest Consecutive Sequence

Medium

Hashing / Set

C++

## QUESTION



Given an unsorted array of integers `nums`, return the length of the longest consecutive elements sequence.



## Example

Input: `nums = [100,4,200,1,3,2]` Output: 4  
 Explanation: The longest consecutive elements sequence is `[1, 2, 3, 4]`. Therefore its length is 4.



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 **$O(n)$  avg.**

SPACE COMPLEXITY

 **$O(n)$** 

PATTERN

**Hashing / Set**

## &lt;/&gt; Solution (C++)

Problem 0128

```
class Solution {
public:
    int longestConsecutive(vector<int>& nums) {
        unordered_set<int> st(nums.begin(),nums.end());
        int ans =0;
        for(int x:st){
            if(st.find(x-1)==st.end()){
                int current =x;
                int count =1;
                while(st.find(current+1)!=st.end()){
                    current++;
                    count++;
                }
                ans=max(ans,count);
            }
        }
        return ans;
    }
};
```

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0189

## #0189

PROBLEM

## 189. Rotate Array

DIFFICULTY

Medium

PATTERN

Math / Simulation

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 189. Rotate Array

Medium

Math / Simulation

C++

## QUESTION



Given an integer array `nums`, rotate the array to the right by `k` steps, where `k` is non-negative.



## Example

Input: `nums = [1,2,3,4,5,6,7]`, `k = 3` Output: `[5,6,7,1,2,3,4]` Explanation: rotate 1 steps to the right: `[7,1,2,3,4,5,6]` rotate 2 steps to the right: `[6,7,1,2,3,4,5]` rotate 3 steps to the right: `[5,6,7,1,2,3,4]`



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 **$O(n)$** 

SPACE COMPLEXITY

 **$O(1)$** 

PATTERN

**Math / Simulation**

&lt;/&gt; Solution (C++)

Problem 0189

```
class Solution {
public:
    void rotate(vector<int>& nums, int k) {
        k = k % nums.size();
        reverse(nums.begin(), nums.end());
        reverse(nums.begin(), nums.begin() + k);
        reverse(nums.begin() + k, nums.end());
    }
};
```

PROBLEM 0209

## #0209

PROBLEM

## 209. Minimum Size Subarray Sum

DIFFICULTY

Medium

PATTERN

Arrays / Implementa...

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 209. Minimum Size Subarray Sum

Medium

Arrays / Implementation

C++

## QUESTION

- ✓ Given an array of positive integers `nums` and a positive integer `target`, return the minimal length of a subarray whose sum is greater than or equal to `target`. If there is no such subarray, return 0 instead.

## Example

Input: `target = 7, nums = [2,3,1,2,4,3]` Output: 2  
 Explanation: The subarray `[4,3]` has the minimal length under the problem constraint.

## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 **$O(n)$** 

SPACE COMPLEXITY

 **$O(1)$** 

PATTERN

**Arrays / Implementation**

## &lt;/&gt; Solution (C++)

Problem 0209

```
class Solution {
public:
    int minSubArrayLen(int target, vector<int>& nums) {
        int l=0;
        int sum=0;
        int len = INT_MAX;
        for(int r=0;r<nums.size();++r){
            sum += nums[r];
            while(sum>=target){
                len = min(r-l+1,len);
                sum-=nums[l];
                ++l;
            }
        }
        if(len == INT_MAX) return 0;
        return len;
    }
};
```

## #0215

PROBLEM

### 215. Kth Largest Element in an Array

DIFFICULTY

Medium

PATTERN

Arrays / Implementa...

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0215

# 215. Kth Largest Element in an Array

Medium

Arrays / Implementation

C++

## QUESTION

- ✓ Given an integer array `nums` and an integer `k`, return the `k`th largest element in the array.

### Example

Input: `nums = [3,2,1,5,6,4]`, `k = 2` Output: 5

### Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

**$O(n)$**



SPACE COMPLEXITY

**$O(1)$**



PATTERN

**Arrays / Implementation**

## </> Solution (C++)

Problem 0215

```
class Solution {
public:
    int findKthLargest(vector<int>& nums, int k) {
        priority_queue<int, vector<int>, greater<int>> pq;
        for(int x:nums){
            pq.push(x);
            if(pq.size()>k){
                pq.pop();
            }
        }
        return pq.top();
    }
};
```

## #0239

PROBLEM

### 239. Sliding Window Maximum

DIFFICULTY

Hard

PATTERN

Two Pointers

LANGUAGE

C++

#### QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

PROBLEM 0239

# 239. Sliding Window Maximum

Hard

Two Pointers

C++

## QUESTION

- ✓ You are given an array of integers `nums`, there is a sliding window of size `k` which is moving from the very left of the array to the very right. You can only see the `k` numbers in the window. Each time the sliding window moves right by one position.

### Example

Input: `nums = [1,3,-1,-3,5,3,6,7]`, `k = 3`  
Output: `[3,3,5,5,6,7]` Explanation: Window position Max ----- [1 3 -1] -3  
5 3 6 7 3 1 [3 -1 -3] 5 3 6 7 3 1 3 [-1 -3 5] 3  
6 7 5 1 3 -1 [-3 5 3] 6 7 5 1 3 -1 -3 [5 3 6] 7  
6 1 3 -1 -3 5 [3 6 7] 7

### Visual Representation

INPUT

PATTERN

OUTPUT

✓ TIME COMPLEXITY  
 **$O(n)$**

✓ SPACE COMPLEXITY  
 **$O(1)$**

✓ PATTERN  
**Two Pointers**

## </> Solution (C++)

Problem 0239

```
class Solution {
public:
    vector<int> maxSlidingWindow(vector<int>& nums, int k) {
        deque<int> dq;
        vector<int> ans;
        for (int r = 0; r < nums.size(); r++) {
            // Remove elements outside the window
            while (!dq.empty() && dq.front() <= r - k) {
                dq.pop_front();
            }
            // Remove smaller elements
            while (!dq.empty() && nums[dq.back()] <= nums[r]) {
                dq.pop_back();
            }
            // Add current index
            dq.push_back(r);
            // Window size is k
            if (r >= k - 1) {
                ans.push_back(nums[dq.front()]);
            }
        }
        return ans;
    }
};
```

PROBLEM 0455

## #0455

PROBLEM

## 455. Assign Cookies

DIFFICULTY

Easy

PATTERN

Sorting

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

20 Sep 2026

Pattern → recall → re-implement

## 455. Assign Cookies

Easy

Sorting

C++

## QUESTION



Assume you are an awesome parent and want to give your children some cookies. But, you should give each child at most one cookie.



## Example

Input:  $g = [1,2,3]$ ,  $s = [1,1]$  Output: 1  
 Explanation: You have 3 children and 2 cookies. The greed factors of 3 children are 1, 2, 3. And even though you have 2 cookies, since their size is both 1, you could only make the child whose greed factor is 1 content. You need to



## Visual Representation

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 $O(n \log n)$ 

SPACE COMPLEXITY

 $O(n) / O(1)$ 

PATTERN

Sorting

## &lt;/&gt; Solution (C++)

Problem 0455

```
class Solution {
public:
    int findContentChildren(vector<int>& g, vector<int>& s) {
        sort(g.begin(), g.end());
        sort(s.begin(), s.end());
        int i = 0;
        int j = 0;
        int count = 0;
        while (i < g.size() && j < s.size()) {
            if (s[j] >= g[i]) {
                count++;
                i++;
                j++;
            }
            else {
                j++;
            }
        }
        return count;
    }
};
```

PROBLEM 3718

#3718

PROBLEM

**4080. Smallest Missing Multiple of K**

DIFFICULTY

Easy

PATTERN

Hashing / Set

LANGUAGE

C++

## QUICK LINKS

- [LeetCode](#)
- [Notes](#)
- [Similar Problems](#)

Last revised

**20 Sep 2026**

Pattern → recall → re-implement

# 4080. Smallest Missing Multiple of K

Easy

Hashing / Set

C++

## QUESTION



Given an integer array `nums` and an integer `k`, return the smallest positive multiple of `k` that is missing from `nums`.

**Example**

Input → Output

**Visual Representation**

INPUT

PATTERN

OUTPUT



TIME COMPLEXITY

 **$O(n)$  avg.**

SPACE COMPLEXITY

 **$O(n)$** 

PATTERN

**Hashing / Set**</> **Solution (C++)**

Problem 3718

```
class Solution {
public:
    int missingMultiple(vector<int>& nums, int k) {
        unordered_set<int> st;
        for(int x : nums){
            st.insert(x);
        }
        for(int i=1;i<=nums.size()+1;i++){
            int x = k*i;
            if(st.find(x)==st.end()){
                return x;
            }
        }
        return -1;
    }
};
```